ure

#1 Database Administration	#3 Enterprise Architecture	#9 Social Protection Systems	#11 NSR Modernisation	#12 Public Sector Digital Transformation

● Day Focus

Day 1 establishes the mental model that underpins the entire programme. Participants must leave understanding WHY PostgreSQL behaves the way it does under concurrent load — not just how to run commands. Spend the most time on WAL and MVCC: every performance, security, and ETL topic in later days assumes this foundation.

Contents

Pre-Session Checklist

- Project the platform URL: drtertseghaanande.com/postgresql-workshop.html
- Log in as Instructor (PIN: AUKTIE2026) — unlocks all days and shows cohort dashboard
- Open two psql terminal windows on the projector — label them 'Terminal A – Lagos' and 'Terminal B – Kano'
- Verify each participant's laptop can reach the platform and run psql
- Direct any late arrivals to the Pre-Session Setup page on the platform
- Check projector resolution — code blocks must be readable from the back of the room

09:30 — Registration & Setup Verification

■ 09:30 Doors open ■ 10:00 Training starts

Ask participants to open the platform and mark Pre-Session Setup complete. Walk the room and fix any PostgreSQL connection issues before 10:00. Do not start the technical content until at least 12 of 14 participants are connected.

■ Section 1 — PostgreSQL Process / Memory / Storage

~60 mins

What to cover

- The **postmaster** forks one backend process per client connection — draw this on the whiteboard: one box for postmaster, arrows to multiple backend boxes, each connected to a client
- Walk through the memory table below — shared_buffers is the single most important tuning parameter (target: 25% of RAM)
- Storage: 8KB pages, tuples inside pages, dead tuples left in place after UPDATE/DELETE — VACUUM reclaims them

Component	Purpose	Config parameter
shared_buffers	Shared page cache (all backends)	shared_buffers
work_mem	Per-sort/hash operation memory	work_mem
wal_buffers	WAL write buffer	wal_buffers
maintenance_work_mem	VACUUM, CREATE INDEX	maintenance_work_mem

● What each memory area actually does

shared_buffers is a pool shared by every connection. When PostgreSQL reads data from disk it stores a copy here so the next query gets it from RAM instead — think of it as a shared filing cabinet the whole office can reach into. It is the most important parameter to tune: the more data you keep cached, the fewer slow disk reads happen. Target 25% of total RAM (e.g. 4 GB on a 16 GB server).

work_mem is the memory budget for a single sort or hash operation — ORDER BY, JOIN, and DISTINCT each get their own slice. With 14 participants connected and each running complex queries, work_mem is multiplied across all concurrent operations. Set it too high and you exhaust RAM. The default (4 MB) is conservative; raise it only when slow sorts are confirmed.

wal_buffers is a small staging area where WAL entries are assembled in memory before being flushed to the WAL log file on disk — like a notepad you scribble on before writing the final record. PostgreSQL tunes this automatically so it rarely needs manual adjustment.

maintenance_work_mem is a separate budget reserved for maintenance operations — VACUUM cleaning up dead tuples and CREATE INDEX building a new index from scratch. Giving maintenance its own allocation keeps it from competing with live queries for memory.

Storage Layout — 8KB Pages, Tuples & Dead Tuples

● Pages, tuples, and dead tuples explained

8KB pages: PostgreSQL never reads individual rows from disk — it reads entire 8-kilobyte blocks called pages. Each page is a fixed-size chunk of a table file on disk. When you query one row, PostgreSQL loads the whole page that contains it into shared_buffers. This is why shared_buffers size matters: if the page is already cached the query is fast; if not, PostgreSQL must go to disk.

Tuples: a tuple is one row in a table. Each page holds as many tuples as fit within 8 KB. Every tuple carries two hidden system columns — xmin (the transaction that created it) and xmax (the transaction that deleted or replaced it). These are the numbers MVCC uses to decide whether a row is visible to a given query.

Dead tuples: when you UPDATE a row, PostgreSQL does not overwrite it — it writes a new version and marks the old one expired by setting xmax. The old version is now a dead tuple: invisible to new queries but still sitting on disk taking up space. PostgreSQL leaves it deliberately because a transaction that started before the update may still need to see it. Once no active transaction can possibly need the old version, VACUUM marks that space as reusable. A table that is updated heavily without regular VACUUM grows larger on disk even if the row count stays the same — this is called table bloat, and Day 6 covers how to detect and fix it.

Demo 1.1 — Query PostgreSQL Settings

Project the terminal and run the query below. Point out: each row is a backend process. Count the rows — that is how many client connections are open. Ask the room: *"What happens to this list when all 14 of you connect simultaneously?"*

```sql
-- View active backend processes
SELECT pid, usename, application_name, state
FROM pg_stat_activity
ORDER BY pid;
```

■ Section 2 — Write-Ahead Logging (WAL)

~40 mins

Key message: WAL delivers Durability. Sequential writes are faster than random writes — that is the engineering insight that makes PostgreSQL fast and crash-safe.

Step 1 — The crash scenario without WAL

● Explain to participants

Imagine PostgreSQL is in the middle of writing a change — updating a household pmt_score. That write is not instant; it involves multiple steps at the hardware level. If power cuts halfway through, the data page on disk is in a half-written, corrupted state — some bytes reflect the old value, some the new one. PostgreSQL has no way to know which parts made it. The database is now unrecoverable without a full restore from backup.

Step 2 — WAL as an insurance log

● The WAL solution

WAL changes the order of operations. Before PostgreSQL touches the actual data page, it first writes a record to the WAL log describing exactly what it is about to do: 'I am going to change this row from value X to value Y.' Only after that log record is safely on disk does PostgreSQL modify the data page. If power cuts now, the data page may still be corrupt — but the WAL log is intact. On restart, PostgreSQL reads the log and replays every committed change, reconstructing the exact state the database should be in. No data loss, no corruption.

The reason this is fast: WAL writes are sequential — always appended to the end of one log file. Sequential disk writes are far faster than the random writes needed to update scattered data pages across many different table files.

Step 3 — Checkpoint

● What a checkpoint does

A checkpoint is a deliberate moment where PostgreSQL flushes every dirty page in shared_buffers to its proper place on disk, then writes a checkpoint record into the WAL marking that point. After a crash, PostgreSQL only needs to replay WAL from the most recent checkpoint — everything before it is already safely on disk. Checkpoints make crash recovery fast and time-bounded.

● Key insight

Sequential WAL writes are much faster than random data-page writes. This is why WAL enables high throughput even on spinning disks.

Key WAL Settings — show and explain on the platform

```sql
SELECT name, setting, unit
FROM pg_settings
WHERE name IN ('wal_level','synchronous_commit','max_wal_size',
'checkpoint_completion_target','wal_compression')
ORDER BY name;
```

Parameter	What to explain to participants
wal_level	Must be at least 'replica' for streaming standby servers
synchronous_commit	'on' = safe; 'off' = 3x write speed but up to 600ms data loss on crash. Use off only for bulk loads, never for payments

max_wal_size	Larger = fewer checkpoints = less I/O spikes, but longer crash recovery
checkpoint_completion_target	0.9 = spread I/O over 90% of the interval. Ask: why would we want checkpoints to take longer?
wal_compression	Free CPU-for-I/O trade. Almost always worth enabling on modern hardware

synchronous_commit — the parameter participants ask about most

● How to explain the trade-off

With synchronous_commit = on (the default), when a transaction commits, PostgreSQL waits until the WAL record has been physically written to disk before telling the application 'commit succeeded.' The data is safe even if the server dies a millisecond later.

With synchronous_commit = off, PostgreSQL responds immediately without waiting for the WAL record to reach disk — roughly 3x faster, but if the server crashes within the next ~600 milliseconds, the most recent commits may be lost.

For NSR: payment records and beneficiary enrolments must always use on — losing a committed payment is unacceptable. Bulk analytics imports and staging table loads can use off because if they fail, you simply re-run the import.

checkpoint_completion_target = 0.9 — ask the room this question

● Ask: "Why would we want checkpoints to take longer?"

This feels counterintuitive — so it makes a good discussion question.

When a checkpoint fires, PostgreSQL must write potentially thousands of dirty pages to disk. If it writes them all at once, there is a sudden spike of disk I/O — queries slow down and the system feels sluggish for several seconds.

checkpoint_completion_target = 0.9 tells PostgreSQL to spread those writes over 90% of the interval between checkpoints rather than doing them all at once. If checkpoints fire every 5 minutes, PostgreSQL takes up to 4.5 minutes to write the dirty pages — a small continuous trickle in the background rather than one large burst.

Result: smooth, consistent I/O instead of periodic spikes. For the NSR where caseworkers expect consistent response times, this is the correct setting.

■ Section 3 — Multi-Version Concurrency Control (MVCC)

~40 mins

● Most important section of the day

Every Day 3 (security), Day 6 (performance), and Day 7 (ETL) concept assumes participants understand MVCC. Do not rush this section.

Delivery sequence

Step 1 — The problem: two caseworkers, one record

● Set the scene for participants

In the NSR, multiple caseworkers across different states are connected to the same database simultaneously. A caseworker in Lagos opens a household record to review it. At the same moment a caseworker in Kano tries to update the pmt_score on that same household. Both are hitting the same row at the same time.

What could go wrong? If both write simultaneously with no coordination, one write could overwrite the other — data is lost. If someone reads while a write is halfway through, they see a partially updated row — corrupted data.

Step 2 — The naive solution: locks (and why they fail)

● Why locks alone are not the answer

The obvious fix is locks: before anyone reads or writes a row, they claim a lock and nobody else can touch it until it is released.

The problem: readers block writers and writers block readers. A caseworker in Lagos simply viewing a household — running a SELECT — holds a read lock. A caseworker in Kano who needs to update that same household has to wait until Lagos finishes reading. In a system with 14 users all querying constantly, everything grinds to a halt waiting for locks to be released. Response times become unpredictable and caseworkers experience freezes.

Step 3 — The MVCC solution: new versions instead of overwrites

● How MVCC eliminates the problem

PostgreSQL never modifies a row in place. When a row is updated, PostgreSQL writes a brand new version of the row alongside the old one. The old version is marked with an expiry transaction ID (xmax) but left physically on disk. The new version gets its own creation transaction ID (xmin). Each active transaction sees whichever version was current at the moment that transaction started.

Result: readers never block writers and writers never block readers. The Lagos caseworker's SELECT reads the version that existed when her transaction began. The Kano caseworker's UPDATE creates a new version without touching what Lagos is reading. Both proceed simultaneously with no waiting.

MVCC concept	What it means in plain terms
xmin	Hidden column on every tuple recording the transaction ID that created this row version. A query can only see a row if its xmin transaction had committed before the reader's snapshot was taken.
xmax	Hidden column recording the transaction ID that deleted or replaced this version. A row with xmax set is expired — invisible to transactions that started after that deletion committed. Zero means the row is still live.
Snapshot isolation	Each transaction gets a consistent point-in-time view of the database, fixed at the moment BEGIN was executed. Subsequent commits by others do not bleed into the current transaction's view.
Dead tuple	The old row version left behind after an UPDATE. It has xmax set so it is expired, but it is still physically on disk taking up space — left there in case an older open transaction still needs to see it.
VACUUM	The cleanup process that scans tables and marks dead tuples as reusable space once no active transaction can possibly need them. Without VACUUM, dead tuples accumulate and the table file grows — this is table bloat.

Demo 1.5 — MVCC Two-Terminal Visibility

● Instructor note — run this setup BEFORE the session starts

> The schema and sample data must exist before you attempt the demo. Steps 1 and 2 below should be completed during your pre-session preparation, not in front of participants. Steps 3 and 4 are the live demo.

Step 1 — Create the NSR schema and tables (pre-session)

Open one psql terminal and connect: psql -U postgres

```sql
-- Create the schema
CREATE SCHEMA nsr;

CREATE TABLE nsr.states (
state_code CHAR(2) PRIMARY KEY,
state_name TEXT NOT NULL UNIQUE
);

CREATE TABLE nsr.lgas (
lga_id SERIAL PRIMARY KEY,
state_code CHAR(2) NOT NULL REFERENCES nsr.states(state_code),
lga_name TEXT NOT NULL
);

CREATE TABLE nsr.households (
household_id BIGSERIAL PRIMARY KEY,
household_code TEXT NOT NULL UNIQUE,
lga_id INT NOT NULL REFERENCES nsr.lgas(lga_id),
head_name TEXT NOT NULL,
pmt_score NUMERIC(8,3),
enumeration_round SMALLINT,
is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```

Step 2 — Insert sample data (pre-session)

```sql
INSERT INTO nsr.states VALUES ('LA','Lagos'), ('KN','Kano');

INSERT INTO nsr.lgas (state_code, lga_name)
VALUES ('LA','Ikeja'), ('KN','Kano Municipal');

-- Insert the household used in the demo
INSERT INTO nsr.households (household_code, lga_id, head_name, pmt_score)
VALUES ('HH-LA-001', 1, 'Fatima Yusuf', 28.5);

-- Confirm it is there — you should see pmt_score = 28.5
SELECT household_id, household_code, head_name, pmt_score
```

```
FROM nsr.households;
```

Step 3 — Open a second terminal window (pre-session)

Open a second Command Prompt or PowerShell window — do not close the first. Connect to PostgreSQL in the second window: psql -U postgres. Place both terminals side by side on the projector screen. Label them visually so the room can follow which is which.

Step 4 — Run the live demo

● Critical: use REPEATABLE READ in Terminal A

PostgreSQL defaults to READ COMMITTED isolation — each statement gets its own fresh snapshot. Under READ COMMITTED, Terminal A's second SELECT would see Terminal B's committed change (42.5) immediately, ruining the demo.

You must start Terminal A's transaction with ISOLATION LEVEL REPEATABLE READ. This fixes the snapshot for the entire transaction, so Terminal A sees a consistent view regardless of what Terminal B commits in between. If you forget this and the demo shows 42.5 on the second SELECT, see the Troubleshooting table below.

Terminal A — Lagos caseworker

Run BEGIN with REPEATABLE READ and the SELECT. Then stop — do not COMMIT or type anything else. Tell the room: 'Terminal A has an open transaction. Its snapshot is fixed at 28.5 — right now.'

sql

```
-- IMPORTANT: specify REPEATABLE READ so the snapshot is fixed for the whole transaction
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT household_id, head_name, pmt_score
FROM nsr.households
WHERE household_code = 'HH-LA-001';
-- You see pmt_score = 28.5. Leave this transaction OPEN.
```

Terminal B — Kano caseworker

Switch to Terminal B. Run the UPDATE and COMMIT. Tell the room: 'Terminal B committed. The new value 42.5 is now in the database.'

sql

```
BEGIN;
UPDATE nsr.households
SET pmt_score = 42.5
WHERE household_code = 'HH-LA-001';
COMMIT;
```

The reveal — back to Terminal A

● Ask the room before running the query

"Terminal B committed 42.5. If I run the same SELECT again in Terminal A right now — what value will I see?" Pause. Let participants answer. Then run it.

sql

```
-- Re-run the same SELECT in Terminal A
SELECT household_id, head_name, pmt_score
FROM nsr.households
WHERE household_code = 'HH-LA-001';
-- Terminal A still shows 28.5 — not 42.5.
```

Terminal A still shows 28.5. This is snapshot isolation: Terminal A's view was fixed at BEGIN. Terminal B's commit happened after that snapshot — so it is outside Terminal A's view entirely.

Close Terminal A's transaction and re-check

sql

```
-- In Terminal A: close the transaction
COMMIT;

-- Now run the SELECT again
SELECT household_id, head_name, pmt_score
FROM nsr.households
WHERE household_code = 'HH-LA-001';
-- Now Terminal A sees 42.5 — a fresh snapshot sees the committed change.
```

Now Terminal A sees 42.5. A new transaction opens a fresh snapshot and sees the world as it is today — including Terminal B's committed change.

Moment	What to say
After Terminal A's first SELECT	'Terminal A has an open transaction. Its snapshot is fixed right now at 28.5.'
After Terminal B COMMITs	'Terminal B committed. 42.5 is now in the database.'
Before Terminal A's second SELECT	'What value will Terminal A see now?' — pause for answers.
After Terminal A still shows 28.5	'Why? Because Terminal A's snapshot was taken at BEGIN. Terminal B's commit is invisible to it.'
After Terminal A COMMITs and sees 42.5	'New transaction, new snapshot — now it sees the world as it is today.'

Problem during demo	Fix
Terminal A shows 42.5 on second SELECT (should show 28.5)	You used plain BEGIN instead of REPEATABLE READ. ROLLBACK in Terminal A, reset value to 28.5, then restart from Step 4 using: BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Value keeps changing between SELECTs with no UPDATE in between	Another terminal session is still open with an uncommitted or re-run UPDATE. Go to every open terminal and run ROLLBACK. Then reset pmt_score to 28.5 and restart.
schema 'nsr' already exists	Run: DROP SCHEMA nsr CASCADE; then repeat Step 1
relation does not exist	Wrong database — run: \c postgres then retry
duplicate key on households	Run: TRUNCATE nsr.households RESTART IDENTITY CASCADE; then re-insert

duplicate key on states or lgas	Run: TRUNCATE nsr.lgas RESTART IDENTITY CASCADE; TRUNCATE nsr.states RESTART IDENTITY CASCADE; then re-insert all data from Step 2
Second terminal asks for password	Use the same password as the first terminal

■ Section 4 — Normalisation & NSR Schema Design

~50 mins

Delivery sequence

- Show the problem of a flat file: state_name repeated on every household row — 10 million rows, 10 million copies of 'Lagos'
- 3NF rule: non-key columns depend only on the primary key, not on each other
- Walk through the schema DDL — use the blue info boxes on the platform to explain each design decision

Core NSR Tables DDL

```sql
CREATE TABLE nsr.states (state_code CHAR(2) PRIMARY KEY, state_name TEXT NOT NULL UNIQUE);

CREATE TABLE nsr.lgas (
lga_id SERIAL PRIMARY KEY,
state_code CHAR(2) NOT NULL REFERENCES nsr.states(state_code),
lga_name TEXT NOT NULL
);

CREATE TABLE nsr.households (
household_id BIGSERIAL PRIMARY KEY,
household_code TEXT NOT NULL UNIQUE,
lga_id INT NOT NULL REFERENCES nsr.lgas(lga_id),
head_name TEXT NOT NULL,
pmt_score NUMERIC(8,3),
enumeration_round SMALLINT,
is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE nsr.individuals (
individual_id BIGSERIAL PRIMARY KEY,
household_id BIGINT NOT NULL REFERENCES nsr.households(household_id) ON DELETE CASCADE,
nin CHAR(11) UNIQUE CHECK (nin IS NULL OR nin ~ '^[A-Z]{2}[0-9]{7}[A-Z]{2}$'),
full_name TEXT NOT NULL,
dob DATE CHECK (dob > '1900-01-01' AND dob <= CURRENT_DATE),
gender CHAR(1) CHECK (gender IN ('M','F','O'))
```

```
);
```

Design decision	Why it matters
BIGSERIAL on individuals	NSR can reach tens of millions of rows — SERIAL overflows at ~2.1 billion
ON DELETE CASCADE	Removing a duplicate household cascades to its members — no orphaned rows
NIN regex CHECK	Enforces NIMC format at DB level — not bypassable by application code
dob range CHECK	Rejects pre-1900 dates and future dates — catches field enumeration errors
NULLABLE nin	Children may not yet have a NIN — NULL is valid; UNIQUE still allows multiple NULLs

Demo 1.7 — Live Normalisation & FK Violation

```sql
-- Insert sample states
INSERT INTO nsr.states VALUES ('LA','Lagos'),('KN','Kano'),('AB','Abuja');

-- Insert sample LGAs
INSERT INTO nsr.lgas (state_code, lga_name)
VALUES ('LA','Ikeja'),('LA','Eti-Osa'),('KN','Kano Municipal');

-- Demonstrate FK enforcement — this will error
INSERT INTO nsr.lgas (state_code, lga_name) VALUES ('XX','Invalid State');
-- Expected: ERROR: insert or update on table lgas violates foreign key constraint
```

Show the error to the room. The database itself is enforcing referential integrity — not application code that can be bypassed or contain bugs.

Exercises & Knowledge Check

Direct participants to the Exercises tab on Day 1 in the platform. Three exercises are available:

Exercise	Level	What to look for
1 — Identify normalisation problems in a flat NSR dataset	Basic	Can they spot repeated data and transitive dependencies?
2 — Write CREATE TABLE DDL for a new NSR table	Intermediate	Correct FK, CHECK constraints, BIGSERIAL, NOT NULL
3 — Explain WAL and MVCC in writing (NSR context)	Advanced	The written explanation is the real diagnostic — if they cannot explain it in their own words they need to revisit

● Walk the room

The advanced exercise (written explanation) is the real diagnostic. Any participant who cannot explain the WAL → MVCC → VACUUM triad in their own words needs more time on those sections before Day 2 begins.

After exercises, open the Knowledge Check tab. Use the quiz scores as a room-temperature check. Any question with widespread wrong answers is a signal to revisit that concept at the start of Day 2.

Day 1 Closing Message

● Suggested closing

"Everything we do this week — the security model, the performance tuning, the ETL pipelines — sits on top of what you learned today. WAL means your data survives a power cut. MVCC means your caseworkers never block each other. The schema you built today is the one you will query, secure, tune, and migrate for the rest of the programme."

Mark Day 1 complete using the Mark Complete button on the platform. Remind participants that Day 2 begins at 09:30 and covers SQL Fundamentals — they should ensure their PostgreSQL connection is working before they arrive.

Common Participant Questions

Q: How much RAM should shared_buffers get?

A: Start at 25% of total RAM. Tune upward if cache hit rate (pg_stat_database) is below 99%.

Q: When should I set synchronous_commit = off?

A: Bulk analytics loads and staging table ingestion only — never for payment or audit records.

Q: Does VACUUM run automatically?

A: Yes — autovacuum runs in the background. It can fall behind on very busy tables. Day 6 covers how to detect table bloat and adjust autovacuum settings.

Q: Can we skip 3NF and just use flat tables?

A: Yes, and you pay the price in update anomalies, storage waste, and broken referential integrity. The NSR at scale cannot afford that.

Q: What happens to the old row version after a COMMIT?

A: It becomes a dead tuple — invisible to new transactions but still occupying disk space. VACUUM (or autovacuum) reclaims it.

Q: Why does Terminal A still see the old pmt_score after Terminal B commits?

A: Two reasons work together. First, Terminal A was started with REPEATABLE READ isolation, so its snapshot is fixed at BEGIN and does not change for the life of the transaction. Second, PostgreSQL MVCC writes a new row version for the update rather than overwriting the old one — so the old version is still physically present on disk for Terminal A to read. Under the default READ COMMITTED level, Terminal A would see 42.5 immediately on the next SELECT, which is why the demo must use REPEATABLE READ.